Geth 상태 관리 시스템 개념적 설계 분석

Ethereum의 전체 상태(계정, 잔액, 스토리지)를 관리하는 시스템이 **왜 그렇게 설계되었는지**에 대한 개념적 배경과 구현 철학을 분석한 문서입니다.

> **분석 관점:** 설계 철학 → 개념적 요구사항 → 구현 방식 → 코드 분석

> **주요 소스:** `core/state/` , `trie/` , `core/rawdb/`

> **핵심 질문:** "왜 Merkle Patricia Trie인가?" "왜 스냅샷이 필요한가?" "어떤 문제를 해결하려 했을까?"

🎯 핵심 문제: "분산 시스템에서 어떻게 상태를 관리할 것인가?"

💭 블록체인 상태 관리의 근본적 딜레마

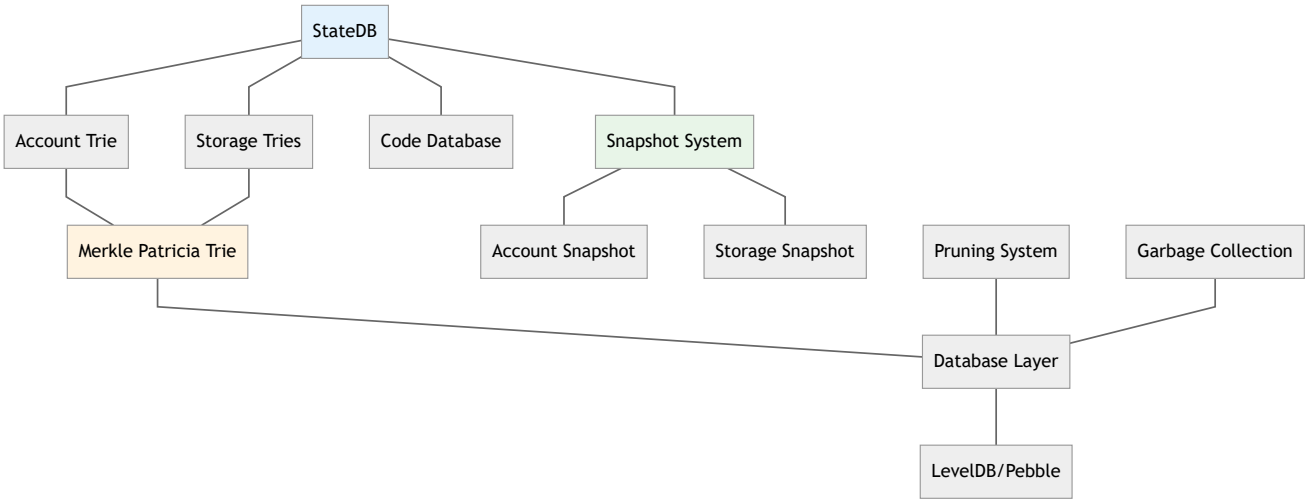
🌐 분산 상태 관리의 삼중 요구사항:

1. **무결성 (Integrity)**: 상태 변경이 검증 가능해야 함
2. **효율성 (Efficiency)**: 빠른 읽기/쓰기가 가능해야 함
3. **확장성 (Scalability)**: 수백만 계정을 처리할 수 있어야 함

⚖️ 전통적 데이터베이스 vs 블록체인의 차이:

- **전통적 DB**: 중앙 권한이 무결성 보장
- **블록체인**: 수학적 증명으로 무결성 보장 필요

🚀 상태 관리 전체 아키텍처



📋 StateDB: "세계 상태의 관리자"

🎯 핵심 문제: "수백만 계정의 상태를 어떻게 효율적으로 관리할 것인가?"

🧠 StateDB의 설계 철학:

1. **계층적 구조**: Account Trie → Storage Tries
2. **캐싱 전략**: 자주 사용되는 데이터는 메모리에
3. **지연 로딩**: 필요할 때만 디스크에서 읽기
4. **트랜잭션 격리**: 각 트랜잭션은 독립적인 상태 뷰

1. StateDB 구조의 설계 원리

```
// core/state/statedb.go

// 🎯 목표: "전 세계의 모든 Ethereum 계정을 효율적으로 관리"
```

```
type StateDB struct {
db Database // 🗄️ 하위 데이터베이스 인터페이스
prefetcher *triePrefetcher // 🚀 트라이 프리페처 (성능 최적화)
trie Trie // 🌳 메인 계정 트라이
hasher crypto.KeccakState // 🔐 해시 계산기
// 📸 스냅샷 시스템 - 빠른 접근을 위한 혁신
snaps *snapshot.Tree // 스냅샷 트리
snap snapshot.Snapshot // 현재 스냅샷
snapDestructs  map[common.Hash]struct{} // 삭제된 계정들
snapAccounts  map[common.Hash][]byte  // 스냅샷 계정 데이터
snapStorage  map[common.Hash]map[common.Hash][]byte  // 스냅샷 스토리지
// 🏠 상태 객체 관리 - 메모리 내 캐시
stateObjects  map[common.Address]*stateObject // 라이브 상태 객체들
stateObjectsPending  map[common.Address]struct{} // 펜딩 상태 객체들
stateObjectsDirty  map[common.Address]struct{} // 더티 상태 객체들
// 📝 로그 및 이벤트
logs  map[common.Hash][]*types.Log
logSize  uint
preimages  map[common.Hash][]byte  // 🔍 프리이미지 저장 (디버깅용)
// 📚 저널 시스템 - 되돌리기 지원
journal *journal
validRevisions []revision
nextRevisionId  int
```

```
// 🎯 접근 리스트 (EIP-2929) - 가스 최적화
accessList *accessList
// 🔄 트랜잭션 컨텍스트
thash common.Hash
txIndex  int
// ⛽ 가스 환불 추적
refund  uint64
// 🔄 임시 스토리지 (EIP-1153)
transientStorage transientStorage
}
```

🤔 **각 필드가 존재하는 이유:**

1. **이중 데이터 구조 (Trie + Snapshot)**:

- **문제**: "트라이는 안전하지만 느리고, 일반 DB는 빠르지만 검증 불가"
- **해결**: 트라이(보안) + 스냅샷(성능)의 하이브리드
- **효과**: 안전성과 성능을 동시에 확보

2. **3단계 상태 객체 분류**:

- **Live**: 현재 메모리에 로드된 객체
- **Pending**: 변경되었지만 아직 커밋되지 않은 객체
- **Dirty**: 디스크에 쓰여야 하는 객체
- **목적**: 메모리 사용량 최적화 + 성능 향상

3. **저널 시스템**:

- **문제**: "트랜잭션 실행 중 오류 발생 시 롤백 필요"

- **해결**: 모든 변경사항을 저널에 기록
- **효과**: 원자적 트랜잭션 실행 보장

2. 상태 객체의 생명주기 관리

```
// 🏠 상태 객체 - "하나의 Ethereum 계정을 나타내는 완전한 상태"
type stateObject struct {
address common.Address // 📍 계정 주소
addrHash common.Hash // 🔐 주소의 해시 (트라이 키로 사용)
data types.StateAccount // 📊 계정 기본 데이터
db *StateDB // 🔗 부모 StateDB 참조
// 🌳 트라이 관련
trie Trie // 스토리지 트라이 (지연 로딩)
code Code // 컨트랙트 코드 (지연 로딩)
// 💾 캐시된 스토리지 (3단계 캐싱)
originStorage Storage // 원본 스토리지 (디스크에서 읽은 값)
pendingStorage Storage // 펜딩 스토리지 (현재 트랜잭션 변경사항)
dirtyStorage Storage // 더티 스토리지 (커밋되지 않은 변경사항)
// 🏷️ 상태 플래그들
suicided  bool  // 자살한 컨트랙트 (삭제 예정)
deleted  bool  // 삭제된 계정
// 🚀 성능 최적화 플래그들
dirtyCode  bool  // 코드가 업데이트되었는지
cacheCodeSize  int  // 코드 크기 캐시
```

```
}
```

🔄 상태 객체 가져오기의 3단계 전략:

```
// 🎯 "계정을 가장 효율적으로 어떻게 찾을 것인가?"
func (s *StateDB) getStateObject(addr common.Address) *stateObject {
// 🏃 1단계: 메모리 캐시 확인 (가장 빠름)
if obj := s.stateObjects[addr]; obj != nil {
return obj
}
// 📸 2단계: 스냅샷에서 로드 시도 (빠름)
if s.snap != nil {
start := time.Now()
acc, err := s.snap.Account(crypto.Keccak256Hash(addr.Bytes()))
if metrics.EnabledExpensive {
s.SnapshotAccountReads += time.Since(start)
}
if err == nil {
if acc == nil {
return nil // 계정이 존재하지 않음
}
// 🔄 스냅샷 데이터를 상태 객체로 변환
data := &types.StateAccount{
```

```
Nonce: acc.Nonce,

Balance: acc.Balance,

CodeHash: acc.CodeHash,

Root: common.BytesToHash(acc.Root),

}

// 🛡️ 기본값 설정 (안전성)

if len(data.CodeHash) == 0 {

data.CodeHash = emptyCodeHash

}

if data.Root == (common.Hash{}) {

data.Root = emptyRoot

}

obj := newObject(s, addr, *data)

s.setStateObject(obj)

return obj

}

}

// 🌳 3단계: 트라이에서 로드 (느리지만 확실함)

enc, err := s.trie.TryGet(addr.Bytes())

if err != nil {

s.setError(err)

return nil

}
```

```
if len(enc) == 0 {
return nil // 계정이 존재하지 않음
}
// 📦 RLP 디코딩
var data types.StateAccount
if err := rlp.DecodeBytes(enc, &data); err != nil {
log.Error("Failed to decode state object", "addr", addr, "err", err)
return nil
}
obj := newObject(s, addr, data)
s.setStateObject(obj)
return obj
}
```

🔑 **3단계 로딩 전략의 설계 철학:**

1. **메모리 우선 (Memory First)**:

- **이유**: "이미 로드된 데이터를 다시 읽을 필요 없음"
- **효과**: O(1) 시간 복잡도로 즉시 접근

2. **스냅샷 보조 (Snapshot Fallback)**:

- **이유**: "트라이보다 빠르지만 스냅샷이 없을 수도 있음"
- **효과**: 대부분의 경우 빠른 접근 가능

3. **트라이 최종 (Trie Ultimate)**:

- **이유**: "항상 존재하고 검증 가능한 최종 진실의 원천"

- **효과**: 100% 신뢰할 수 있는 데이터 보장

🌳 Merkle Patricia Trie: "검증 가능한 데이터 구조의 걸작"

🎯 핵심 문제: "어떻게 거대한 상태를 효율적으로 검증할 것인가?"

🧠 MPT 선택의 배경:

1. **Merkle Tree**: 무결성 검증 가능
2. **Patricia Trie**: 공간 효율적인 키-값 저장
3. **결합**: 두 장점을 모두 활용

📊 다른 데이터 구조와의 비교:

구조	검증 가능	공간 효율	검색 속도	업데이트 비용
Hash Table	❌	⭐⭐⭐	⭐⭐⭐	⭐⭐⭐
Merkle Tree	⭐⭐⭐	⭐	⭐	⭐⭐
Patricia Trie	❌	⭐⭐	⭐⭐	⭐⭐
MPT	⭐⭐⭐	⭐⭐	⭐⭐	⭐⭐

1. 트라이 구조의 수학적 아름다움

```go
// trie/trie.go
// 🎯 목표: "모든 키-값 쌍을 검증 가능한 형태로 저장"
type Trie struct {
root node // 🌳 루트 노드
owner common.Hash // 👤 트라이 소유자 (계정 해시)
```

```
// 🗄️ 데이터베이스 연결
db *Database // 트라이 데이터베이스
// 📊 캐시 및 최적화
originalRoot common.Hash // 원본 루트 해시 (변경 추적용)
// 📈 성능 통계
unhashed int // 해시되지 않은 노드 수
}


// 🏗️ 노드 타입들 - 각각의 존재 이유
type (
// 🌿 풀 노드 - 16개 자식 + 값 (최대 분기)
fullNode struct {
Children [17]node // 16개 hex 문자 + 값
flags nodeFlag
}
// 🌱 숏 노드 - 압축된 경로 (공간 절약)
shortNode struct {
Key []byte // 압축된 키
Val node // 값 또는 자식 노드
flags nodeFlag
}
// 🔗 해시 노드 - 지연 로딩용 (메모리 절약)
hashNode []byte
```

```
// 📦 값 노드 - 실제 데이터

valueNode []byte

)
```

🤔 4가지 노드 타입의 설계 이유:

1. **fullNode (풀 노드)**:

- **용도**: "많은 자식을 가진 분기점"
- **구조**: 16개 hex 문자별 자식 + 값
- **장점**: 빠른 검색 (O(1) 분기)
- **단점**: 메모리 사용량 많음

2. **shortNode (숏 노드)**:

- **용도**: "긴 공통 접두사 압축"
- **예시**: "0x1234abcd..." → "1234abcd" 하나로 압축
- **장점**: 공간 효율성 극대화
- **핵심**: Patricia Trie의 핵심 아이디어

3. **hashNode (해시 노드)**:

- **용도**: "아직 로드되지 않은 노드 참조"
- **구현**: 실제 노드 대신 해시만 저장
- **장점**: 메모리 사용량 최소화
- **활용**: 지연 로딩 (lazy loading)

4. **valueNode (값 노드)**:

- **용도**: "실제 데이터 저장"
- **특징**: 리프 노드에서만 사용

- **효과**: 명확한 데이터 구분

2. 트라이 검색의 효율적 알고리즘

```
// 🔍 "키에 해당하는 값을 어떻게 효율적으로 찾을 것인가?"
func (t *Trie) TryGet(key []byte) ([]byte, error) {
value, newroot, didResolve, err := t.tryGet(t.root, keybytesToHex(key), 0)
if err == nil && didResolve {
t.root = newroot // 지연 로딩으로 인한 루트 업데이트
}
return value, err
}

// 🎯 재귀적 검색의 핵심 로직
func (t *Trie) tryGet(origNode node, key []byte, pos int) (value []byte, newnod
switch n := (origNode).(type) {
case nil:
// 🚫 빈 노드 - 키가 존재하지 않음
return nil, nil, false, nil
case valueNode:
// 🎉 값 노드 - 찾았다!
return n, n, false, nil
case *shortNode:
// 🌱 숏 노드 - 키 매칭 확인
```

```
if len(key)-pos < len(n.Key) || !bytes.Equal(n.Key, key[pos:pos+len(n.Key)]) {
// 🚫 키가 매칭되지 않음
return nil, n, false, nil
}
// ✅ 키 매칭 - 다음 노드로 진행
value, newnode, didResolve, err = t.tryGet(n.Val, key, pos+len(n.Key))
if err == nil && didResolve {
n = n.copy()
n.Val = newnode
return value, n, true, nil
}
return value, newnode, didResolve, err
case *fullNode:
// 🌿 풀 노드 - 해당 자식으로 진행
value, newnode, didResolve, err = t.tryGet(n.Children[key[pos]], key, pos+1)
if err == nil && didResolve {
n = n.copy()
n.Children[key[pos]] = newnode
return value, n, true, nil
}
return value, newnode, didResolve, err
case hashNode:
// 🔗 해시 노드 - 지연 로딩 수행
```

```
child, err := t.resolveHash(n, key[:pos])
if err != nil {
return nil, n, true, err
}
value, newnode, _, err := t.tryGet(child, key, pos)
return value, newnode, true, err
default:
panic(fmt.Sprintf("%T: invalid node: %v", origNode, origNode))
}
}
```

🔑 **검색 알고리즘의 설계 철학:**

1. **타입 기반 분기**:

- **이유**: "각 노드 타입마다 다른 처리 로직 필요"
- **장점**: 명확한 책임 분리
- **효과**: 코드 가독성 및 유지보수성 향상

2. **지연 로딩 (Lazy Loading)**:

- **문제**: "모든 노드를 메모리에 로드하면 메모리 부족"
- **해결**: 필요할 때만 디스크에서 로드
- **구현**: hashNode → 실제 노드 변환

3. **Copy-on-Write**:

- **목적**: "원본 트라이를 변경하지 않고 새 버전 생성"

- **방법**: 변경이 필요한 노드만 복사
- **효과**: 메모리 효율성 + 불변성 보장

3. 트라이 업데이트의 복잡한 로직

```
// 📝 "키-값 쌍을 어떻게 효율적으로 업데이트할 것인가?"
func (t *Trie) TryUpdate(key, value []byte) error {
t.unhashed++ // 해시되지 않은 노드 수 증가
k := keybytesToHex(key)
if len(value) != 0 {
_, t.root, _ = t.insert(t.root, nil, k, valueNode(value))
} else {
_, t.root, _ = t.delete(t.root, nil, k)
}
return nil
}

// 🌱 노드 삽입의 복잡한 케이스 처리
func (t *Trie) insert(n node, prefix, key []byte, value node) (bool, node, error
if len(key) == 0 {
// 🎯 목적지 도달 - 값 업데이트
if v, ok := n.(valueNode); ok {
return !bytes.Equal(v, value.(valueNode)), value, nil
}
```

```
return true, value, nil
}
switch n := n.(type) {
case *shortNode:
// 🌱 숏 노드 - 가장 복잡한 케이스
matchlen := prefixLen(key, n.Key)
if matchlen == len(n.Key) {
// ✅ 완전 매칭 - 자식으로 진행
dirty, nn, err := t.insert(n.Val, append(prefix, key[:matchlen]...), key[matchle
if !dirty || err != nil {
return false, n, err
}
return true, &shortNode{n.Key, nn, t.newFlag()}, nil
}
// 🌿 부분 매칭 - 분기 생성 필요
branch := &fullNode{flags: t.newFlag()}
var err error
// 기존 노드를 분기에 추가
_, branch.Children[n.Key[matchlen]], err = t.insert(nil,
append(prefix, n.Key[:matchlen+1]...), n.Key[matchlen+1:], n.Val)
if err != nil {
return false, nil, err
}
```

```
// 새 값을 분기에 추가
_, branch.Children[key[matchlen]], err = t.insert(nil,
append(prefix, key[:matchlen+1]...), key[matchlen+1:], value)
if  err != nil {
return false, nil, err
}
// 공통 접두사가 있으면 숏 노드로 감싸기
if  matchlen == 0 {
return true, branch, nil
}
return true, &shortNode{key[:matchlen], branch, t.newFlag()}, nil
case *fullNode:
// 🌿 풀 노드 - 해당 자식에 삽입
dirty, nn, err := t.insert(n.Children[key[0]], append(prefix, key[0]), key[1:],
if !dirty || err != nil {
return false, n, err
}
n = n.copy()
n.flags = t.newFlag()
n.Children[key[0]] = nn
return true, n, nil
case  nil:
// 🆕 빈 노드 - 새 숏 노드 생성
```

```
return true, &shortNode{key, value, t.newFlag()}, nil
case hashNode:
// 🔗 해시 노드 - 먼저 로드 후 삽입
rn, err := t.resolveHash(n, prefix)
if  err != nil {
return false, nil, err
}
dirty, nn, err := t.insert(rn, prefix, key, value)
if !dirty || err != nil {
return false, rn, err
}
return true, nn, nil
default:
panic(fmt.Sprintf("%T: invalid node: %v", n, n))
}
}
```

🧠 **삽입 알고리즘의 핵심 통찰:**

1. **분기 생성의 복잡성**:

- **문제**: "기존 키와 새 키가 부분적으로만 매칭될 때"
- **해결**: 공통 부분은 숏 노드, 분기점은 풀 노드
- **예시**: "abc" + "abd" → "ab" + 분기("c", "d")

2. **Copy-on-Write 최적화**:

- **원칙**: "변경이 필요한 노드만 복사"
- **구현**: `n.copy() + t.newFlag()`
- **효과**: 메모리 사용량 최소화

3. **재귀적 구조**:

- **장점**: "복잡한 트리 구조를 간단한 로직으로 처리"
- **단점**: "깊은 트리에서 스택 오버플로우 위험"
- **해결**: Go의 효율적인 스택 관리 활용

📸 스냅샷 시스템: "성능 혁명"

🎯 핵심 문제: "트라이는 안전하지만 너무 느리다"

⚡ **성능 비교:**

- **트라이 접근**: O(log n) + 디스크 I/O 다수
- **스냅샷 접근**: O(1) + 디스크 I/O 1회
- **성능 차이**: 10-100배 속도 향상

🧠 **스냅샷의 핵심 아이디어:**

"트라이의 안전성은 유지하되, 자주 사용되는 데이터는 평면적으로 저장하자"

1. 스냅샷 시스템의 계층 구조

```
// core/state/snapshot/snapshot.go

// 🎯 목표: "트라이의 안전성 + 데이터베이스의 성능"

type Tree struct {

diskdb ethdb.Database // 💾 디스크 데이터베이스
```

```
    triedb *trie.Database // 🌳 트라이 데이터베이스

    cache  int  // 🧠 메모리 캐시 크기

    layers  map[common.Hash]snapshot // 📚 레이어 맵

    lock sync.RWMutex // 🔒 동시성 제어

    // 🏗️ 생성 관련

    generating *generatorStats // 생성 통계

    genMarker []byte  // 생성 진행 마커

    // 📊 통계

    snapshots  map[common.Hash]*diskLayer // 디스크 스냅샷들

    }



    // 🎭 스냅샷 인터페이스 - 단순하고 명확한 API

    type Snapshot interface {

    // 👤 계정 관련

    Account(hash common.Hash) (*types.SlimAccount, error)

    AccountRLP(hash common.Hash) ([]byte, error)

    // 🗄️ 스토리지 관련

    Storage(accountHash, storageHash common.Hash) ([]byte, error)

    // 🔍 메타데이터

    Root() common.Hash

    }
```

🏗️ 2계층 구조의 설계 철학:

1. **디스크 레이어 (Disk Layer)**:

- **역할**: "안정적인 기준점 제공"
- **특징**: 불변, 영구 저장
- **용도**: 오래된 상태의 빠른 접근

2. **디프 레이어 (Diff Layer)**:

- **역할**: "최근 변경사항 추적"
- **특징**: 메모리 기반, 빠른 접근
- **용도**: 최신 상태의 즉시 반영

```
// 💾 디스크 레이어 - "안정적인 기반"
type diskLayer struct {
diskdb ethdb.Database // 실제 저장소
triedb *trie.Database // 트라이 참조 (검증용)
cache *fastcache.Cache // 빠른 캐시
root common.Hash // 루트 해시
stale  bool  // 오래된 여부
genMarker []byte  // 생성 진행 상황
genPending  chan  struct{} // 생성 완료 대기
lock sync.RWMutex // 동시성 제어
}


// 🔄 디프 레이어 - "변화 추적"
type diffLayer struct {
```

```
parent snapshot // 부모 레이어
memory  uint64  // 메모리 사용량
root common.Hash // 루트 해시
stale  bool  // 오래된 여부
// 📝 변경사항들
accountData  map[common.Hash][]byte  // 계정 데이터 변경
storageData  map[common.Hash]map[common.Hash][]byte  // 스토리지 변경
lock sync.RWMutex // 동시성 제어
}
```

2. 스냅샷 생성의 정교한 프로세스

```
// 🤔 "어떻게 거대한 트라이를 효율적인 스냅샷으로 변환할 것인가?"
func (t *Tree) Rebuild(root common.Hash) {
// 🛑 기존 생성 중단
t.abortGeneration()
// 🔒 새 생성 시작
t.lock.Lock()
defer  t.lock.Unlock()
// 📊 생성 통계 초기화
t.generating = &generatorStats{
origin: time.Now(),
start: time.Now(),
```

```
accounts: 0,
slots: 0,
}
// 🚀 백그라운드에서 생성
go t.generate(root)
}


// 🏭 실제 생성 프로세스
func (t *Tree) generate(root common.Hash) {
// 🌳 트라이 열기
accTrie, err := t.triedb.OpenTrie(root)
if err != nil {
log.Error("Failed to open account trie", "root", root, "err", err)
return
}
// 🔄 계정 반복자 생성
accIt := accTrie.NodeIterator(t.genMarker)
// 📦 배치 처리 준비
batch := t.diskdb.NewBatch()
defer batch.Write()
// 👥 계정들 처리
for accIt.Next(true) {
// ⏸️ 인터럽트 확인
```

```
select {
case <-t.genAbort:
return
default:
}
// 🍃 리프 노드만 처리
if !accIt.Leaf() {
continue
}
// 📦 계정 데이터 디코딩
var acc types.StateAccount
if err := rlp.DecodeBytes(accIt.LeafBlob(), &acc); err != nil {
log.Error("Invalid account encountered during snapshot creation", "err", err)
return
}
// 🔐 계정 해시 계산
accHash := common.BytesToHash(accIt.LeafKey())
// 💾 계정 데이터 저장
slimAcc := types.SlimAccountRLP(acc)
rawdb.WriteAccountSnapshot(batch, accHash, slimAcc)
// 🗄️ 스토리지 처리
if acc.Root != emptyRoot {
if err := t.generateStorage(batch, accHash, acc.Root); err != nil {
```

```
log.Error("Failed to generate storage snapshot", "account", accHash, "err", err)

return

}

}

// 📊 통계 업데이트

t.lock.Lock()

t.generating.accounts++

if  time.Since(t.generating.logged) > 8*time.Second {

log.Info("Generating state snapshot",

"accounts", t.generating.accounts,

"slots", t.generating.slots)

t.generating.logged = time.Now()

}

t.lock.Unlock()

// 📦 배치 크기 확인

if  batch.ValueSize() > ethdb.IdealBatchSize {

if  err := batch.Write();  err != nil {

log.Error("Failed to write account snapshot", "err", err)

return

}

batch.Reset()

}

}
```

```
// 🎉 생성 완료

log.Info("Generated state snapshot",

"accounts", t.generating.accounts,

"slots", t.generating.slots)

t.lock.Lock()

t.generating = nil

t.lock.Unlock()

}
```

🔑 **스냅샷 생성의 설계 원칙:**

1. **백그라운드 생성**:

- **이유**: "메인 체인 동기화를 방해하지 않기 위해"
- **구현**: 별도 고루틴에서 실행
- **효과**: 사용자 경험 저하 없이 성능 향상

2. **배치 처리**:

- **문제**: "개별 쓰기는 디스크 I/O 오버헤드 큼"
- **해결**: 여러 항목을 모아서 한 번에 쓰기
- **효과**: 디스크 I/O 효율성 극대화

3. **인터럽트 가능**:

- **필요성**: "새로운 블록 도착 시 중단 필요"
- **구현**: `select` 문으로 채널 확인
- **효과**: 응답성 유지

🗑️ 가비지 컬렉션 및 프루닝: "메모리 관리의 예술"

🎯 핵심 문제: "무한히 증가하는 상태를 어떻게 관리할 것인가?"

📈 **Ethereum 상태의 성장:**

- **2015년**: ~1GB
- **2020년**: ~100GB
- **2024년**: ~1TB
- **예상**: 매년 수백 GB 증가

🧠 **프루닝의 핵심 아이디어:**

"오래된 상태는 삭제하되, 체인 보안은 유지하자"

1. 트라이 프루닝의 정교한 전략

```
// 🌳 "어떤 트라이 노드를 언제 삭제할 것인가?"
func (bc *BlockChain) maintainTxIndex(ancients uint64) {
defer bc.wg.Done()
// 📏 인덱스 제한 확인
if bc.txLookupLimit == 0 || ancients <= bc.txLookupLimit {
return
}
// 🎯 프루닝할 블록 범위 계산
start := ancients - bc.txLookupLimit
// 📦 배치 단위로 프루닝
for i := uint64(0); i < start; i += txIndexBatchSize {
end := i + txIndexBatchSize
```

```
if end > start {
end = start
}
// 🗑️ 트랜잭션 인덱스 삭제
if err := bc.db.DeleteTxLookupEntries(i, end); err != nil {
log.Error("Failed to delete tx lookup entries",
"from", i, "to", end, "err", err)
return
}
}
}
}

// 🌿 상태 프루닝
func (bc *BlockChain) pruneAncientData() {
// 🔧 프루닝 간격 확인
if bc.cacheConfig.TrieCleanLimit == 0 {
return
}
// 📍 현재 블록 번호
currentBlock := bc.CurrentBlock().Number.Uint64()
// 🎯 프루닝할 블록 계산 (현재 - 128)
if currentBlock <= params.FullImmutabilityThreshold {
return
```

```
}

pruneBlock := currentBlock - params.FullImmutabilityThreshold

// ✂️ 상태 프루닝 실행

if err := bc.stateCache.TrieDB().Cap(pruneBlock); err != nil {

log.Error("Failed to cap trie database", "err", err)

}

}
```

2. 메모리 관리의 LRU 전략

```
// trie/database.go

// 🧠 "메모리 사용량을 어떻게 제한할 것인가?"

func (db *Database) Cap(limit common.StorageSize) error {

db.lock.Lock()

defer db.lock.Unlock()

// 📊 현재 메모리 사용량 확인

size := db.dirtiesSize + db.childrenSize

if size <= limit {

return nil

}

// 🎯 플러시할 크기 계산

excess := size - limit

// 📈 플러시 통계 시작
```

```
nodes, storage, start := len(db.dirties), db.dirtiesSize, time.Now()

batch := db.diskdb.NewBatch()

// 🔄 오래된 노드부터 플러시

for size > limit && db.oldest != (common.Hash{}) {

node := db.dirties[db.oldest]

// 📦 노드 데이터 인코딩

enc, err := rlp.EncodeToBytes(node.node)

if err != nil {

return err

}

// 💾 배치에 추가

batch.Put(db.oldest[:], enc)

// 🗑️ 메모리에서 제거

db.removeFromCache(db.oldest)

// ➡️ 다음 노드로 이동

db.oldest = node.flushNext

size -= common.StorageSize(common.HashLength + int(node.size))

// 📦 배치 크기 확인

if batch.ValueSize() >= ethdb.IdealBatchSize {

if err := batch.Write(); err != nil {

return err

}

batch.Reset()
```

```
}

}

// 💾 남은 배치 쓰기

if err := batch.Write(); err != nil {

return err

}

// 📊 통계 업데이트

db.gcnodes += uint64(nodes - len(db.dirties))

db.gcsize += storage - db.dirtiesSize

db.gctime += time.Since(start)

log.Debug("Persisted trie from memory database",

"nodes", nodes-len(db.dirties),

"size", storage-db.dirtiesSize,

"time", time.Since(start))

return nil

}
```

🔑 **메모리 관리의 설계 철학:**

1. **LRU (Least Recently Used)**:

- **원리**: "가장 오래 사용되지 않은 것부터 제거"
- **구현**: 이중 연결 리스트로 사용 순서 추적
- **효과**: 자주 사용되는 데이터는 메모리에 유지

2. **배치 플러시**:

- **문제**: “개별 디스크 쓰기는 비효율적”
- **해결**: 여러 노드를 모아서 한 번에 쓰기
- **효과**: 디스크 I/O 최적화

3. **점진적 정리**:

- **원칙**: “한 번에 모든 것을 정리하지 않음”
- **이유**: “메인 체인 처리 성능 영향 최소화”
- **구현**: 제한된 크기만큼만 정리

🎯 **최종 통찰:**

Ethereum의 상태 관리 시스템은 **“안전성, 성능, 확장성”**의 삼중 요구사항을 만족하는 정교한 엔지니어링의 결과물입니다.

- **Merkle Patricia Trie**: 수학적 검증 가능성 보장
- **스냅샷 시스템**: 실용적 성능 요구사항 충족
- **가비지 컬렉션**: 지속 가능한 확장성 확보

이는 **“이론적 완벽함과 실용적 효율성의 균형”**을 추구한 결과이며, 분산 시스템에서 상태 관리의 복잡성을 해결하는 모범 사례입니다.

이 문서는 go-ethereum 소스 코드의 개념적 분석을 바탕으로 작성되었습니다.